

▼ Column Info

Columns

Well Identification and Location

WELL: Unique well identifier.
DEPTH_MD: Measured depth (MD) in meters (vertical distance down the wellbore).
X_LOC, Y_LOC: Easting and Northing coordinates (well's horizontal position, likely UTM).
Z_LOC: Depth in a vertical coordinate system (elevation relative to sea level, different from DEPTH_MD).

Geological Information (Ground Truth)

GROUP: Broader geological grouping of formations.
FORMATION: Specific geological formation at a given depth (finer subdivision than GROUP).
FORCE_2020_LITHOFACIES_LITHOLOGY: Target variable: lithology (rock type) at each depth (categorical, with codes for different lithologies).
FORCE_2020_LITHOFACIES_CONFIDENCE: Confidence in the assigned lithology label (important for evaluating model performance).
Well Log Measurements (Features)
CALI: Caliper log (borehole diameter).
RSA: Shallow resistivity log.
RMD: Medium resistivity log.
RDEP: Deep resistivity log.
RHOB: Bulk density log.
GR: Gamma-ray log.
SGR: Spectral gamma ray log.
NPHI: Neutron porosity log.
PEF: Photoelectric factor log.
DTC: Sonic/acoustic log (sound wave travel time).
SP: Spontaneous potential log.
BS: Bit size (diameter of the drill bit).
ROP: Rate of penetration (drilling speed).
DTS: Shear wave velocity log.
DCAL: Difference between caliper and bit size.
DRHO: Density correction.
MUDWEIGHT: Density of the drilling mud.
RMIC: Micro-resistivity log.
ROPA: Processed/interpreted resistivity measurement.
RXO: Resistivity of the flushed zone.

Dataset Use Cases

This dataset is suited for developing and testing machine learning models for lithology prediction. The various well log measurements provide valuable input features, while the FORCE_2020_LITHOFACIES_LITHOLOGY column serves as the target variable. The well location and geological information (GROUP, FORMATION) can provide additional context for analysis. The target code corresponds to lithologies that are mainly encountered in the North Sea reservoirs. The dataset is highly imbalanced, so keep this in mind.

Code	Lithology
30000	Sandstone
65030	Sandstone/Shale
65000	Shale
80000	Marl
74000	Dolomite
70000	Limestone
70032	Chalk
88000	Halite
86000	Anhydrite
99000	Tuff

```
90000 Coal
93000 Basement
```

Module & Data Loading

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
import os
```

```
import kagglehub
path = kagglehub.dataset_download("kawatgiomkar/lithology")
```

```
Downloading from https://www.kaggle.com/api/v1/datasets/download/kawatgiomkar/lithology?dataset\_version\_number=1...
100%|██████████| 86.8M/86.8M [00:01<00:00, 82.3MB/s]Extracting files...
```

```
os.listdir(path)
```

```
['CSV_train.csv']
```

```
df = pd.read_csv(path + '/CSV_train.csv', sep=';')
df.shape
```

```
(1170511, 29)
```

```
df=df[df['WELL']=='25/4-5']
df.shape
```

```
(23005, 29)
```

```
df.head(1)
```

	WELL	DEPTH_MD	X_LOC	Y_LOC	Z_LOC	GROUP	FORMATION	CALI	RSHA	RMED	...	ROP	DTS
309361	25/4-5	853.302805	454469.53125	6603730.0	-828.002808	HORDALAND GP.	Utsira Fm.	19.158823	NaN	1.163803	...	NaN	NaN

```
1 rows x 29 columns
```

Data Preprocessing

Null Values

```
print(df.isnull().sum())
```

WELL	0
DEPTH_MD	0
X_LOC	0
Y_LOC	0
Z_LOC	0
GROUP	0
FORMATION	0
CALI	0
RSHA	16778
RMED	9
RDEP	0
RHOB	0
GR	0
SGR	23005
NPHI	0

```

PEF                17226
DTC                 56
SP                 23005
BS                  0
ROP                 23005
DTS                 23005
DCAL                0
DRHO                26
MUDWEIGHT           23005
RMIC                16849
ROPA                23005
RXO                 23005
FORCE_2020_LITHOFACIES_LITHOLOGY 0
FORCE_2020_LITHOFACIES_CONFIDENCE 3
dtype: int64

```

SGR, SP, ROP, MUDWEIGHT, DTS, RXO, ROPA Are not used entirely in this well

RSHA, PEF, RMIC -> 80% null

> Columns To Drop

↳ 4 cells hidden

✓ Duplicated Values

```
df_clean.duplicated().sum()
```

```
np.int64(0)
```

✓ Data Mis-Type

```
df_clean.info()
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 23005 entries, 309361 to 332365
Data columns (total 19 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   WELL                                  23005 non-null  object
 1   DEPTH_MD                             23005 non-null  float64
 2   X_LOC                                23005 non-null  float64
 3   Y_LOC                                23005 non-null  float64
 4   Z_LOC                                23005 non-null  float64
 5   GROUP                                23005 non-null  object
 6   FORMATION                             23005 non-null  object
 7   CALI                                  23005 non-null  float64
 8   RMED                                  23005 non-null  float64
 9   RDEP                                  23005 non-null  float64
10  RHOB                                  23005 non-null  float64
11  GR                                    23005 non-null  float64
12  NPHI                                  23005 non-null  float64
13  DTC                                    23005 non-null  float64
14  BS                                    23005 non-null  float64
15  DCAL                                  23005 non-null  float64
16  DRHO                                  23005 non-null  float64
17  FORCE_2020_LITHOFACIES_LITHOLOGY      23005 non-null  int64
18  FORCE_2020_LITHOFACIES_CONFIDENCE     23005 non-null  float64
dtypes: float64(15), int64(1), object(3)
memory usage: 3.5+ MB

```

✓ Log Plot

```

df_clean = df_clean.sort_values('DEPTH_MD')

# (tracks)
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 12), sharey=True)

# (well name)

```

```

well_name = df_clean['WELL'].iloc[0]
fig.suptitle(f'Well Log Plot - {well_name}', fontsize=14, fontweight='bold')

# -----

# Track 1: Depth Track
ax1.set_ylabel('DEPTH (m)')
ax1.invert_yaxis() # Invert y-axis so depth increases downward
ax1.grid(True, alpha=0.3)

# Add depth ticks on the left
ax1.set_ylim(df_clean['DEPTH_MD'].max(), df_clean['DEPTH_MD'].min()) # Inverted
ax1.get_xaxis().set_visible(False) # Hide x-axis for depth track

# -----

# Track 2: Gamma Ray (GR)
ax2.plot(df_clean['GR'], df_clean['DEPTH_MD'], color='green', linewidth=0.8)
ax2.set_xlabel('GR (API)')
ax2.set_title('Gamma Ray')
ax2.grid(True, alpha=0.3)
ax2.invert_yaxis()
ax2.set_xlim(0, 450)
ax2.fill_betweenx(df_clean['DEPTH_MD'], 0, df_clean['GR'],
                  where=(df_clean['GR'] > 0),
                  color='green', alpha=0.3)

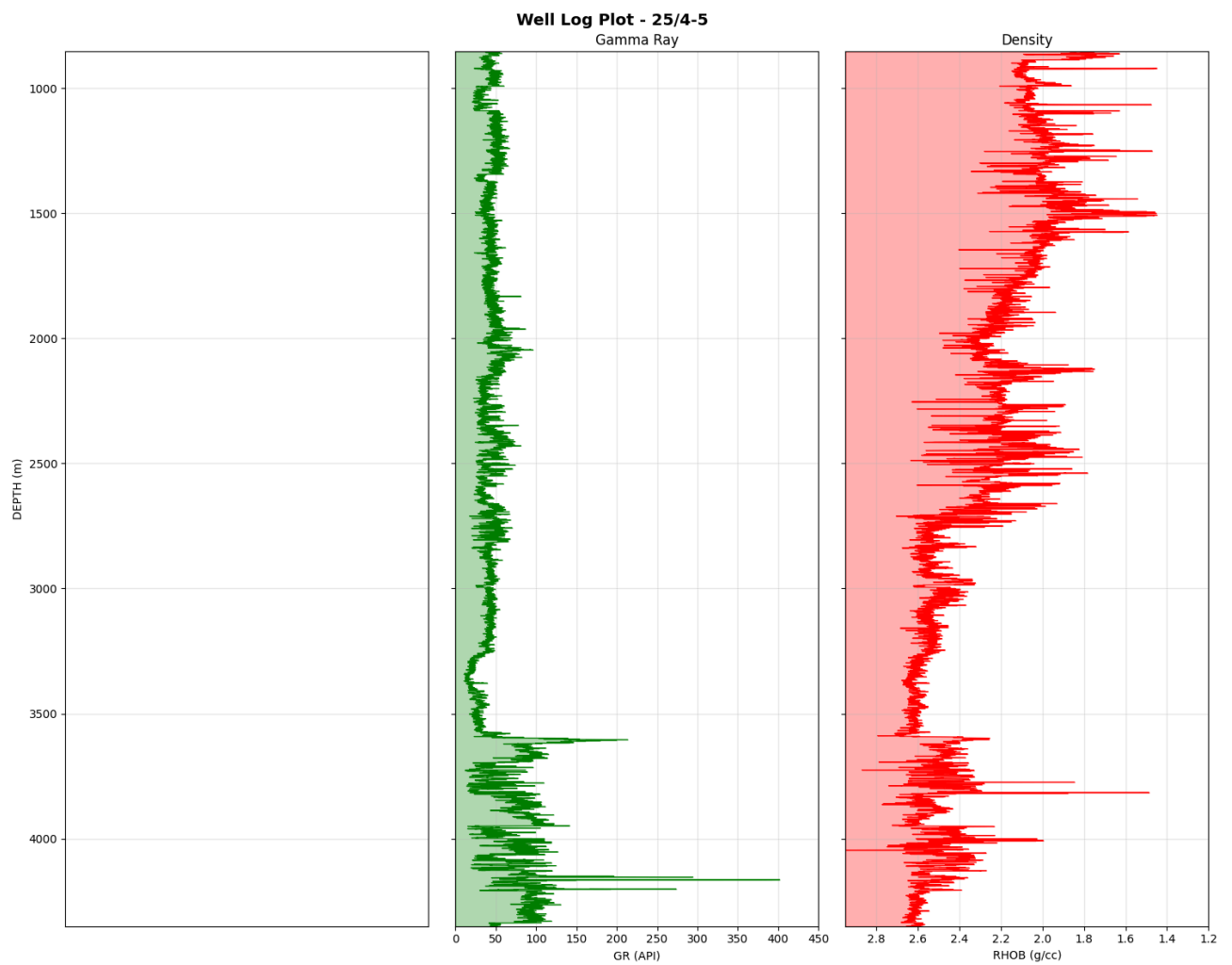
# -----

# Track 3: Density (RHOB)
ax3.plot(df_clean['RHOB'], df_clean['DEPTH_MD'], color='red', linewidth=0.8)
ax3.set_xlabel('RHOB (g/cc)')
ax3.set_title('Density')
ax3.grid(True, alpha=0.3)
ax3.invert_yaxis()
ax3.set_xlim(1.2, 2.95)
# Invert x-axis for density (common practice - lower density to the right)
ax3.invert_xaxis()

ax3.fill_betweenx(df_clean['DEPTH_MD'], df_clean['RHOB'], 2.95, # Fill from curve to right edge
                  color='red', alpha=0.3)

# Adjust layout
plt.subplots_adjust(wspace=0.05) # Reduce space between tracks
plt.tight_layout()
plt.show()

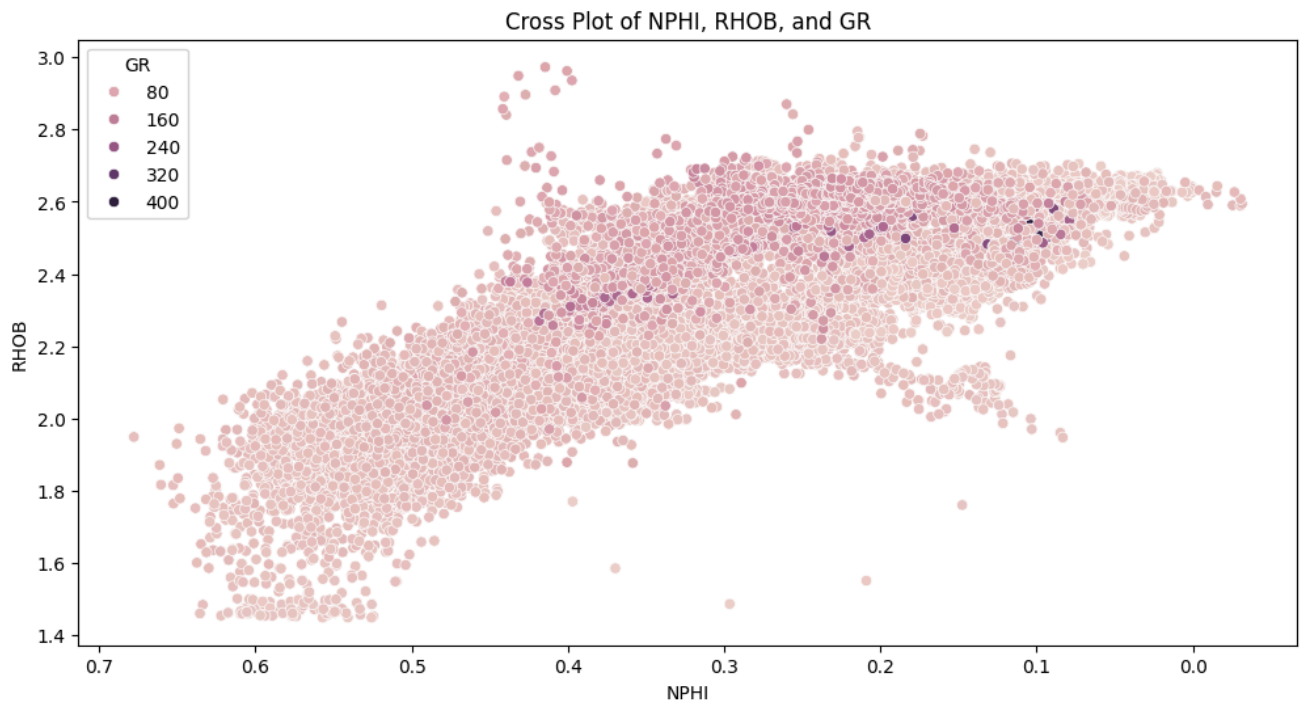
```



✓ Cross Plot

```
plt.figure(figsize=(12, 6))
sns.scatterplot(df_clean, x='NPHI', y='RHOB', hue='GR')
# plt.gca().invert_yaxis()
plt.gca().invert_xaxis()

plt.title('Cross Plot of NPHI, RHOB, and GR')
plt.show()
```



Modeling

Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
# Define features and target
X = df_clean[['DEPTH_MD', 'GR', 'RHOB', 'NPHI']]

y = df_clean['FORCE_2020_LITHOFACIES_LITHOLOGY']

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

RandomForestClassifier

```
RandomForestClassifier(random_state=42)
```

```
y_pred = rf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy}')
```

Accuracy: 0.9171919148011302

```
print("Classification Report", classification_report(y_test, y_pred))
```

Classification Report		precision	recall	f1-score	support
30000	0.92	0.95	0.94	909	
65000	0.92	0.94	0.93	1777	
65030	0.81	0.76	0.78	596	
70000	0.94	0.88	0.91	446	
80000	0.96	0.98	0.97	824	
90000	0.83	0.71	0.77	7	

	99000	0.84	0.90	0.87	42
accuracy				0.92	4601
macro avg	0.89	0.87	0.88		4601
weighted avg	0.92	0.92	0.92		4601

```
df_slice = df_clean[(df_clean['DEPTH_MD'] >= 2000) & (df_clean['DEPTH_MD'] <= 2500)]
```

```
X_slice = df_slice[['DEPTH_MD', 'GR', 'RHOB', 'NPHI']]
y_slice_pred = rf.predict(X_slice)
df_slice['PREDICTED_LITHOLOGY'] = y_slice_pred
```

```
lithology_dict = {
    30000: 'Sandstone',
    65030: 'Sandstone/Shale',
    65000: 'Shale',
    80000: 'Marl',
    74000: 'Dolomite',
    70000: 'Limestone',
    70032: 'Chalk',
    88000: 'Halite',
    86000: 'Anhydrite',
    99000: 'Tuff',
    90000: 'Coal',
    93000: 'Basement'
}

df_slice['LITHOLOGY_NAME'] = df_slice['FORCE_2020_LITHOFACIES_LITHOLOGY'].map(lithology_dict)
df_slice['PREDICTED_LITHOLOGY_NAME'] = df_slice['PREDICTED_LITHOLOGY'].map(lithology_dict)

print(df_slice[['DEPTH_MD', 'LITHOLOGY_NAME', 'PREDICTED_LITHOLOGY_NAME']].head())
```

	DEPTH_MD	LITHOLOGY_NAME	PREDICTED_LITHOLOGY_NAME
316906	2000.142805	Tuff	Tuff
316907	2000.294805	Tuff	Tuff
316908	2000.446805	Tuff	Tuff
316909	2000.598805	Tuff	Tuff
316910	2000.750805	Tuff	Tuff

```
lithology_colors = {
    'Sandstone': '#FDEE00',
    'Sandstone/Shale': '#808000',
    'Shale': '#355E3B',
    'Marl': '#AAB7B8',
    'Dolomite': '#D7BDE2',
    'Limestone': '#F0F8FF',
    'Chalk': '#848482',
    'Halite': '#FADBD8',
    'Anhydrite': '#ABB2B9',
    'Tuff': '#90EE90',
    'Coal': '#000000',
    'Basement': '#800020'
}

fig, axes = plt.subplots(ncols=2, nrows=1, figsize=(10, 15), sharey=True)

# Plot Actual Lithology
axes[0].plot(df_slice['DEPTH_MD'], df_slice['DEPTH_MD'], color='white', linewidth=0)

# Dummy plot to get depth range
current_lithology = None
start_depth = None

for idx, row in df_slice.iterrows():
    if current_lithology is None:
        current_lithology = row['LITHOLOGY_NAME']
        start_depth = row['DEPTH_MD']

    elif row['LITHOLOGY_NAME'] != current_lithology:
        end_depth = row['DEPTH_MD']
        axes[0].axhspan(start_depth, end_depth, facecolor=lithology_colors.get(current_lithology, 'CCCCCC'), alpha=0.9)
```

```

current_lithology = row['LITHOLOGY_NAME']
start_depth = row['DEPTH_MD']

# Handle the last lithology section
if current_lithology is not None:
    axes[0].axhspan(start_depth, df_slice['DEPTH_MD'].iloc[-1], facecolor=lithology_colors.get(current_lithology, '#CCCCCC'))

axes[0].set_title('Actual Lithology')
axes[0].set_ylabel('Depth (m)')
axes[0].invert_yaxis()
axes[0].grid(True, alpha=0.3)
axes[0].set_xlim(0, 1) # Dummy x-limit
axes[0].set_xticks([]) # Hide x-axis ticks

# Plot Predicted Lithology
axes[1].plot(df_slice['DEPTH_MD'], df_slice['DEPTH_MD'], color='white', linewidth=0) # Dummy plot to get depth range

current_predicted_lithology = None
start_depth_pred = None

for idx, row in df_slice.iterrows():
    if current_predicted_lithology is None:
        current_predicted_lithology = row['PREDICTED_LITHOLOGY_NAME']
        start_depth_pred = row['DEPTH_MD']

    elif row['PREDICTED_LITHOLOGY_NAME'] != current_predicted_lithology:
        end_depth_pred = row['DEPTH_MD']
        axes[1].axhspan(start_depth_pred, end_depth_pred, facecolor=lithology_colors.get(current_predicted_lithology, '#CCCCCC'))
        current_predicted_lithology = row['PREDICTED_LITHOLOGY_NAME']
        start_depth_pred = row['DEPTH_MD']

# Handle the last lithology section
if current_predicted_lithology is not None:
    axes[1].axhspan(start_depth_pred, df_slice['DEPTH_MD'].iloc[-1], facecolor=lithology_colors.get(current_predicted_lithology, '#CCCCCC'))

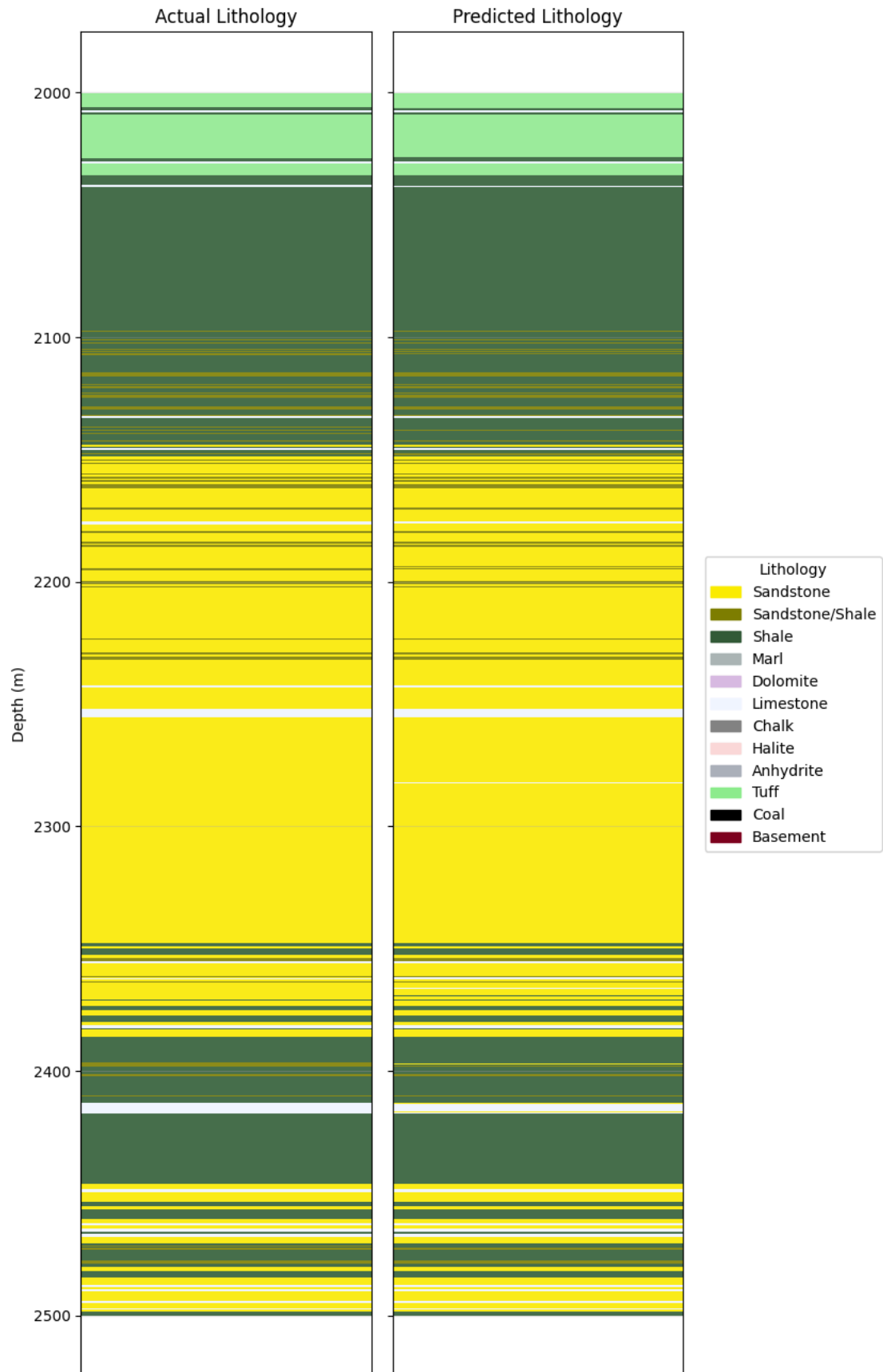
axes[1].set_title('Predicted Lithology')
axes[1].grid(True, alpha=0.3)
axes[1].set_xlim(0, 1) # Dummy x-limit
axes[1].set_xticks([]) # Hide x-axis ticks

# Create a legend
legend_handles = [plt.Rectangle((0,0),1,1, color=lithology_colors[name]) for name in lithology_colors.keys()]
axes[1].legend(legend_handles, lithology_colors.keys(), title="Lithology", loc='center left', bbox_to_anchor=(1.05, 0.5))

plt.suptitle(f'Actual vs. Predicted Lithology for Well {df_slice["WELL"].iloc[0]} (2000m-2500m)', y=0.92, fontsize=16)
plt.tight_layout(rect=[0, 0, 0.85, 0.9])
plt.show()

```


Actual vs. Predicted Lithology for Well 25/4-5 (2000m-2500m)



✓ Summary:

Q&A

The visual comparison between the actual and predicted lithology for the depth range of 2000m to 2500m shows a good overall fit, especially for larger lithology blocks.

The Random Forest Classifier successfully identified prominent lithological boundaries and maintained consistency across significant depth intervals.

Areas of good fit include the accurate prediction of dominant lithology types (e.g., Tuff, Shale) in continuous sections and reasonably well-captured transitions between lithology types.